

Building GNFR Spend Visibility on the Ascend Agentic Harness

Why the delivery method — not just the architecture — is on the critical path to the November go/no-go.

PREPARED FOR	GNFR Indirect Spend stakeholders
SUBJECT	AAH as the delivery vehicle for M1-M4
BASIS OF ASSESSMENT	FY2026 roadmap; M1-M4 requirements; agentic architecture deep-dive; the working spend-visibility-mvp build
STATUS	Internal — for discussion

§ 01

The decision in front of us

The GNFR roadmap sets a go/no-go on **20 November** between continuing with the parallel build and standing on the internal one. Between now and then, four capabilities — Spend Visibility, Document Digitization, Sourcing Automation and Invoice Leakage — have to reach UAT, clear AI and Cloud Governance, and be validated end-to-end against the parallel build's outputs.

That compresses an uncomfortable amount into a short window, and it does so under two conditions that are unusual for a data product. First, **governance approval is a hard gate**: the roadmap states plainly that without AI and Cloud Governance sign-off, no AI capability goes live. Second, several decisions the build depends on are **still open** — the categorization taxonomy is unresolved between the Coupa commodity hierarchy and a business taxonomy, the contract extraction approach is undecided, risk-tier thresholds need a working session, and the human-in-the-loop spend threshold has no number yet.

A team can build quickly, or it can build with an audit trail. The argument for AAH is that on this programme those stopped being alternatives some time ago.

§ 02

What the harness actually is

AAH is a delivery framework layered on Claude Code. It takes an ambiguous problem statement through a fixed phase sequence, with autonomous agents doing the work behind deterministic quality gates that a person

cannot wave through. Its output is not only software — it is software plus the record of how every decision inside it was reached.

PHASE 01 Orchestrate init-project · discuss · access* · arch · ux* · plan	PHASE 02 Build build · fix	PHASE 03 Validate deploy
---	---	---------------------------------------

Three things distinguish it from a team simply using an AI coding assistant well. Every discovery answer becomes a **recorded decision** with a stable identifier, the options that were considered, the ones eliminated, and who decided. Every module is built **one at a time in dependency order**, behind gates for tests, standards, security and regression. And **mock-based testing is prohibited** — tests execute against the real running system or they do not count.

§ 03

Where it earns its place on this programme

The case is not that AAH is a good framework in general. It is that this programme's specific, documented constraints happen to line up with what the harness enforces by default.

PROGRAMME REQUIREMENT	WHAT THE HARNESS DOES ABOUT IT	WHERE IT SHOWS UP
AI & Cloud Governance approval blocks every AI capability from going live	Security scanning is a merge gate, not a pre-launch scramble. Critical and high findings block the merge; SAST, secrets, dependency, licence, IaC and SBOM scanning run continuously, with a release-certification step before ship.	Severity policy in core/security/policy.py; project-local semgrep-rules/ and nuclei-templates/ already scaffolded for authz, SQL injection, audit logging and IDOR
Risk tiers and recommendations must be explainable and rule-based, never an AI black box	Every design decision is a registry entry carrying the question asked, the options presented, the options eliminated, the response, its source and its revision history. A traceability matrix links problem statement through decision, spec, feature and test to code.	.aah/discuss/decision-registry.yaml; core/intel/build_traceability.py
Taxonomy, extraction approach, risk thresholds and non-PO matching are all still undecided	Decisions carry status. Open and provisional items are recorded with rationale and built behind a seam, so work proceeds without pretending the question is settled — and reversing later is a decision revision, not archaeology.	The current registry already holds taxonomy: open, data-foundation: provisional, cloud-target: provisional

PROGRAMME REQUIREMENT	WHAT THE HARNESS DOES ABOUT IT	WHERE IT SHOWS UP
Oracle AP totals must reconcile against source records (M1, Story 1.1)	Mock frameworks are banned outright. A reconciliation claim is only satisfied by a test that runs the real service against real data — which is the only kind of test that means anything for a spend product.	NO MOCKS rule in the project's CLAUDE.md; smoke tests assert real aggregate totals through the live ASGI app
Coupa non-prod SSO is unconfigured and blocks discrepancy-flagging tests	Configuration is a contract. A committed .env.example catalogues every variable the system needs, and a boot check fails loudly and completely when any is missing. Environments that do not exist yet are placeholders, not hard-coded assumptions.	config/env_check.py raising ERR_CDR_78_EX_CONFIG; PING dev, stage and prod issuers already catalogued as placeholders
Four capabilities must be validated end-to-end against parallel build outputs	One repeatable pipeline produces the same artefact structure for each capability, so parity assessment compares like with like instead of comparing four teams' conventions.	Identical .aah/ layout per project: PRD, decision registry, specs, features, DAG, test results, audit log
Parallel build offramp with knowledge transfer for in-scope capabilities	The handover package is generated as the work happens rather than assembled afterwards — discovery PRD, decision history, traceability matrix and accumulated codebase intelligence.	.aah/discuss/discuss-prd.md, .aah/codebase-intel/expertise.yaml, phase audit log

§ 04

What the current build already demonstrates

This is not a theoretical assessment. The spend-visibility-mvp repository was taken from a raw problem statement to a running three-workload system, and its own metadata records the elapsed time.

~2 days	From project scaffold to a working system spanning all three workloads, per the manifest's own created and updated timestamps.
3 workloads	A Next.js dashboard with filters, drill-through detail drawer and agent chat panel; a FastAPI service; and a LangGraph recommendation agent — kept as separately deployable units, matching the target architecture.
11 decisions	Recorded in the discovery registry with question, options, source and rationale — alongside five open questions captured explicitly rather than quietly assumed.
4 seams	Databricks ingestion, governance event capture, Redis cache and PING OIDC are all present as configuration-driven boundaries, so live endpoints slot in without a code change.

2 planes	Dynatrace OTLP for infrastructure and LangSmith for agent tracing are wired in and degrade gracefully when unconfigured — the observability posture the deep-dive calls for, present from day one.
1 ledger	A prioritised refactor register that also names what must <i>not</i> be touched: generated artefacts, fixture data, framework metadata. Technical debt is visible and triaged rather than accumulating unremarked.

Critically, the data model the MVP settled on already carries **source provenance**, **managed versus unmanaged coverage** and **data-quality flags** on every record. Those are not incidental fields — they are the direct expression of M1's source precedence rule, contract coverage requirement and data-quality flagging stories. The discovery phase surfaced them because it was made to ask.

§ 05

What it does not solve

Four gaps are worth pricing in before committing, and one of them is on the critical path.

The deploy phase is shaped for AWS

On the critical path. AAH's automated deployment routes target AWS and Google Cloud — Bedrock AgentCore, Amplify, ECS, Cloud Run. Azure appears in connectivity and readiness probing but not as a deployment route, while the target architecture is AKS with API Management and Azure OpenAI. Orchestrate, Build and Validate transfer without modification; deployment either needs an AKS route added to the harness or falls back to standard infrastructure-as-code and your existing pipeline. This is a one-time extension, not a per-capability cost — but it should be funded now rather than discovered in October.

There is no indirect-procurement domain brief

The shipped domain taxonomy covers financial services. A GNFR brief encoding sourcing vocabulary, spend taxonomy structure and compliance expectations would need authoring. Also one-time, and it pays back across all four capabilities rather than only the first.

The MVP proves the shape, not the data

Today's build runs on a small synthetic dataset. It demonstrates the system's structure and the delivery method convincingly; it does not yet demonstrate the Databricks and Unity Catalog path, row-level security by category manager, or reconciliation against Oracle AP. Those are the next vertical slices, and they are where the real risk sits.

The harness does not make business decisions

Taxonomy choice, risk-tier thresholds, the human-in-the-loop spend threshold and non-PO invoice matching still need the working sessions the requirement documents already name. AAH makes the absence of those decisions *visible* and makes reversing them cheap. It does not resolve them, and it should not be presented as though it will.

There is also a cultural cost worth stating plainly: the gates are non-negotiable by design. A team accustomed to iterating freely will feel friction in the first fortnight. That friction is the product, and it is what buys the audit trail — but it is real, and it lands early.

§ 06

Recommendation

01 Adopt AAH across M1–M4, not M1 alone.

The setup cost is largely one-time. Marginal cost falls with each capability, and the parity workstream's end-to-end validation becomes materially cheaper when all four share one artefact structure.

02 Fund the Azure deployment route and a GNFR domain brief now.

Both are one-time investments and both sit on the path to a credible November comparison. Neither is expensive in August; either could be expensive in October.

03 Treat the `.aah/` directory as the contractual knowledge-transfer artefact.

The offramp obligation in the parity workstream is satisfied by artefacts the build produces anyway, rather than by a series of handover sessions after the fact.

04 Route every open decision back through the registry.

Take taxonomy, risk tiers and the HitL threshold to the working sessions the requirements already identify — then record the outcome as a decision revision, so November's review can be shown a decision trail rather than a codebase.

The November assessment will not turn only on whether the internal build works. It will turn on whether we can show *why* it works the way it does — which decisions were taken, on what basis, by whom, and what would change if the business decided otherwise. That evidence is expensive to reconstruct and nearly free to accumulate. The harness accumulates it.